

BASE DE DATOS 2023

UNIVERSIDAD DE ALCALÁ

Escuela Politécnica Superior

Ciencias de la Computación

Grado en Ingeniería en Sistemas de Información

25/12/2023

AUTORES:

Isaac Aaron Marval Hernández 51752280A

Pedro Bustos Sánchez-C 06612815Q

Profesor: Ángel Javier Álvarez Miguel

Universidad de Alcalá

CONTENIDO

base de datos 2023 1

AUTORES: 1

Memoria de los Triggers 3

Trigger de Auditoría para Eventos de Inserción, Modificación y Borrado 3

Gestión Dinámica de Páginas Web en Críticas y Carátulas 6

Gestión Dinámica de Puntuación Media: Creación y Actualización Automática 8

Memoria de Usuarios 10

Modificaciones sobre Python 11

MEMORIA DE LOS TRIGGERS

Trigger de Auditoría para Eventos de Inserción, Modificación y Borrado

Creación de la tabla

```
CREATE TABLE auditoria(
    operation char(1),
    stamp     timestamp,
    userid    text,
    last_date timestamp,
    last_change text,
    tabla    text,
    PRIMARY KEY (operation, stamp)
);
```

Se crea la tabla auditoria que almacenará información sobre las operaciones realizadas en otras tablas.

Los campos incluyen:

- operation: Tipo de operación (I para INSERT, U para UPDATE, D para DELETE).
- stamp: Marca de tiempo de la auditoría.
- userid: Identificación del usuario que realizó la operación.
- last_date: Fecha de la última modificación.
- last_user: Último usuario que realizó la modificación.
- tabla: Nombre de la tabla en la que se realizó la operación.

Creación de la función

```
CREATE OR REPLACE FUNCTION auditoria() RETURNS TRIGGER AS $audit$
BEGIN
  IF TG_OP = 'UPDATE' THEN
    INSERT INTO auditoria(last_date, operation, stamp, userid,
last_change, tabla)
    VALUES (current_timestamp, 'U', now(), current_user, NEW,
TG_TABLE_NAME);
    RETURN NEW;
  ELSIF TG_OP = 'DELETE' THEN
    INSERT INTO auditoria(last_date, operation, stamp, userid,
last_change, tabla)
    VALUES (current_timestamp, 'D', now(), current_user, OLD,
TG_TABLE_NAME);
    RETURN OLD;
```

```
  ELSIF TG_OP = 'INSERT' THEN
    INSERT INTO auditoria(last_date, operation, stamp, userid,
last_change, tabla)
    VALUES (current_timestamp, 'I', now(), current_user, NEW,
TG_TABLE_NAME);
    RETURN NEW;
  END IF;
END;
$audit$ LANGUAGE plpgsql;
```

Se crea una función trigger llamada auditoria() que se activará antes de las operaciones INSERT, UPDATE o DELETE.

La función compara TG_OP (tipo de operación) para determinar qué acción se realizó.

Para operaciones de UPDATE, se registra en la tabla auditoria la nueva información (NEW) junto con la información anterior (OLD).

Para operaciones de DELETE, se registra solo la información anterior (OLD).

Para operaciones de INSERT, se registra solo la nueva información (NEW).

La función devuelve NEW o OLD según la operación realizada.

Creación de los disparadores

```
CREATE TRIGGER peliculas_auditoria BEFORE INSERT OR UPDATE ON "Peliculas"
    FOR EACH ROW EXECUTE FUNCTION auditoria();

CREATE TRIGGER personal_auditoria BEFORE INSERT OR UPDATE ON "Personal"
    FOR EACH ROW EXECUTE FUNCTION auditoria();

CREATE TRIGGER actores_auditoria BEFORE INSERT OR UPDATE ON "Actores"
    FOR EACH ROW EXECUTE FUNCTION auditoria();

CREATE TRIGGER directores_auditoria BEFORE INSERT OR UPDATE ON
"Directores"
    FOR EACH ROW EXECUTE FUNCTION auditoria();

CREATE TRIGGER guionistas_auditoria BEFORE INSERT OR UPDATE ON
"Guionistas"
    FOR EACH ROW EXECUTE FUNCTION auditoria();

CREATE TRIGGER actuan_auditoria BEFORE INSERT OR UPDATE ON "Actuan"
    FOR EACH ROW EXECUTE FUNCTION auditoria();
```

```
CREATE TRIGGER guionizan_auditoria BEFORE INSERT OR UPDATE ON "Guionizan"
    FOR EACH ROW EXECUTE FUNCTION auditoria();

CREATE TRIGGER paginaweb_auditoria BEFORE INSERT OR UPDATE ON "Paginaweb"
    FOR EACH ROW EXECUTE FUNCTION auditoria();

CREATE TRIGGER caratulas_auditoria BEFORE INSERT OR UPDATE ON "Caratulas"
    FOR EACH ROW EXECUTE FUNCTION auditoria();

CREATE TRIGGER criticas_auditoria BEFORE INSERT OR UPDATE ON "Criticas"
    FOR EACH ROW EXECUTE FUNCTION auditoria();

CREATE TRIGGER alojadas_auditoria BEFORE INSERT OR UPDATE ON "Alojadas"
    FOR EACH ROW EXECUTE FUNCTION auditoria();
```

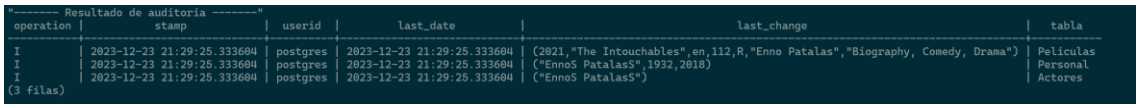
Todos los trigger se ejecutan antes del INSERT o el UPDATE

Pruebas realizadas

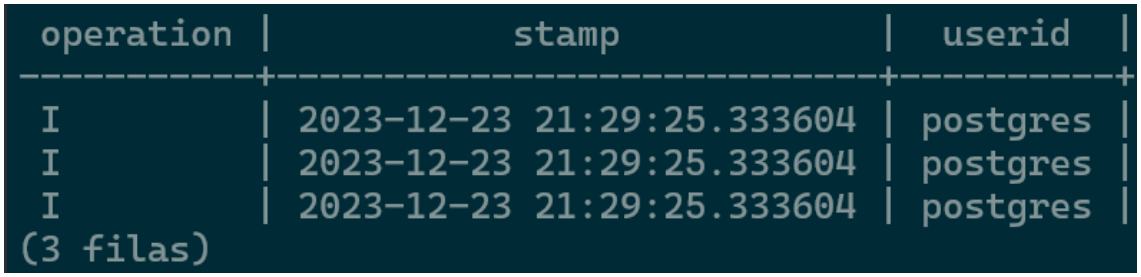
```
INSERT INTO "Caratulas" (nombre_caratula, tamanio, anio_peliculas,
titulo_peliculas, pagina_web_pagina_web)
VALUES ('Hellboy', 'Carátula_mediana', 2004, 'Hellboy',
'https://palomitas.com/películas/hellboy_Carátula_mediana.jpg');

INSERT INTO "Criticas" (anio_peliculas, titulo_peliculas, nombre_critico,
puntuacion, texto_critica, pagina_web_pagina_web)
VALUES (2004, 'Hellboy', 'Roger Ebert', 3.5, 'The movie is entertaining
in its own way, but it''s a long way from the top shelf of recent comic
book movies.', 'https://www.rogerebert.com/reviews/hellboy-2004');
```

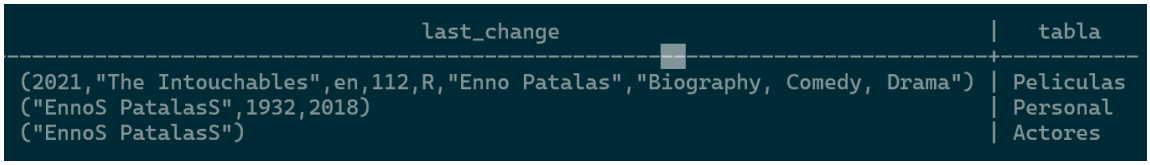
Resultado



```
"------- Resultado de auditoria -------"
 operation |          stamp          | userid |        last_date        |                                                last_change                                                 |   tabla
-----------+-------------------------+--------+-------------------------+------------------------------------------------------------------------------------------------------------+-----------
 I         | 2023-12-23 21:29:25.333604 | postgres | 2023-12-23 21:29:25.333604 | (2021,"The Intouchables",en,112,R,"Enno Patalas","Biography, Comedy, Drama") | Peliculas
 I         | 2023-12-23 21:29:25.333604 | postgres | 2023-12-23 21:29:25.333604 | ("EnnoS PatalasS",1932,2018)                                                                               | Personal
 I         | 2023-12-23 21:29:25.333604 | postgres | 2023-12-23 21:29:25.333604 | ("EnnoS PatalasS")                                                                                         | Actores
(3 filas)
```



```
 operation |          stamp          | userid
-----------+-------------------------+----------
 I         | 2023-12-23 21:29:25.333604 | postgres
 I         | 2023-12-23 21:29:25.333604 | postgres
 I         | 2023-12-23 21:29:25.333604 | postgres
(3 filas)
```



```
                                last_change                                 |   tabla
----------------------------------------------------------------------------+-----------
(2021,"The Intouchables",en,112,R,"Enno Patalas","Biography, Comedy, Drama") | Peliculas
("EnnoS PatalasS",1932,2018)                                                | Personal
("EnnoS PatalasS")                                                          | Actores
```

Gestión Dinámica de Páginas Web en Críticas y Carátulas

Creación de la Función check_webpage:

```
------   Crear funcion para introducir pagina web ------
--CREATE TABLE IF NOT EXISTS public."Paginaweb"

CREATE OR REPLACE FUNCTION check_webpage() RETURNS TRIGGER AS
$pagina_cambio$
BEGIN
    INSERT INTO public."Paginaweb" (pagina_web) VALUES
(NEW.pagina_web_pagina_web);
    RETURN NEW;
END;
$pagina_cambio$ LANGUAGE plpgsql;
```

La función check_webpage se define para ser ejecutada como un trigger antes de la inserción en las tablas "Caratulas" y "Criticas".

En la sección BEGIN...END, se realiza la acción principal: la inserción de la nueva página web (NEW.pagina_web_pagina_web) en la tabla "Paginaweb".

Creación de Triggers

```
CREATE TRIGGER check_webpage_before_insert
BEFORE INSERT ON "Caratulas"
FOR EACH ROW EXECUTE FUNCTION check_webpage();

CREATE TRIGGER check_webpage_before_insert
BEFORE INSERT ON "Criticas"
FOR EACH ROW EXECUTE FUNCTION check_webpage();
```

Se crean dos triggers (check_webpage_before_insert) asociados a las tablas "Caratulas" y "Criticas" que se activan antes de una inserción.

Ambos triggers ejecutan la función check_webpage(), asegurando que la página web se gestione dinámicamente antes de insertar una carátula o una crítica.

Prueba realizada:

```
INSERT INTO "Caratulas" (nombre_caratula, tamanio, anio_peliculas, titulo_peliculas, pagina_web_pagina_web)
VALUES ('Hellboy', 'Carátula_mediana', 2004, 'Hellboy', 'https://palomitas.com/películas/hellboy_Carátula_mediana.jpg');

INSERT INTO "Criticas" (anio_peliculas, titulo_peliculas, nombre_critico, puntuacion, texto_critica, pagina_web_pagina_web)
VALUES (2004, 'Hellboy', 'Roger Ebert', 3.5, 'The movie is entertaining in its own way, but it''s a long way from the top sh
```

Insertamos un dato nuevo e introduce la pagina web en la tabla para cumplir con el FK.

Gestión Dinámica de Puntuación Media: Creación y Actualización Automática

Creación de la Tabla de Puntuación Media

```
------   Puntuacion media de las peliculas ------
CREATE TABLE IF NOT EXISTS public.puntuacion_media
(
    nombre_pelicula text,
    puntuacion_media float
);
```

Se crea una tabla llamada "puntuacion_media" para almacenar la puntuación media de cada película.

La tabla tiene dos columnas: nombre_pelicula para el título de la película y puntuacion_media para la puntuación media.

Creación de la Función

```
CREATE OR REPLACE FUNCTION puntuacion_media() RETURNS TRIGGER AS
$puntuacion_media$
BEGIN
    RAISE NOTICE 'Updating average rating for %', NEW.titulo_peliculas;
    IF EXISTS (SELECT 1 FROM public.puntuacion_media WHERE
nombre_pelicula = NEW.titulo_peliculas) THEN
        UPDATE public.puntuacion_media
        SET puntuacion_media = (SELECT AVG(c.puntuacion)
        FROM public."Criticas" c WHERE c.titulo_peliculas =
NEW.titulo_peliculas AND c.anio_peliculas = NEW.anio_peliculas)
        WHERE nombre_pelicula = NEW.titulo_peliculas;
    ELSE
        INSERT INTO public.puntuacion_media (nombre_pelicula,
puntuacion_media)
        VALUES (NEW.titulo_peliculas, (SELECT AVG(c.puntuacion)
        FROM public."Criticas" c WHERE c.titulo_peliculas =
NEW.titulo_peliculas AND c.anio_peliculas = NEW.anio_peliculas));
    END IF;
```

```
    RAISE NOTICE 'Average rating updated to %', (SELECT puntuacion_media
FROM public.puntuacion_media WHERE nombre_pelicula =
NEW.titulo_peliculas);
    RETURN NULL;
END;
$puntuacion_media$ LANGUAGE plpgsql;
```

Se define la función puntuacion_media que se activa después de cada inserción o actualización en la tabla "Criticas".

La función calcula y actualiza la puntuación media de la película afectada, y luego inserta o actualiza la entrada correspondiente en la tabla "puntuacion_media".

Se utiliza RAISE NOTICE para imprimir mensajes informativos durante el proceso.

Creación del Trigger

```
CREATE TRIGGER puntuacion_media
AFTER INSERT OR UPDATE ON "Criticas"
FOR EACH ROW EXECUTE FUNCTION puntuacion_media();
```

Se crea un trigger llamado puntuacion_media que se activa después de cada inserción o actualización en la tabla "Criticas".

Este trigger ejecuta la función puntuacion_media definida anteriormente.

Pruebas de Puntuación Media

```
INSERT INTO "Criticas" (anio_peliculas, titulo_peliculas, nombre_critico,
puntuacion, texto_critica, pagina_web_pagina_web)
VALUES (2004, 'Hellboy', 'Roger EbertPO', 3.5, 'The movie is entertaining
in its own way, but it''s a long way from the top shelf of recent comic
book movies.', 'https://www.rogerebertO.com/reviews/hellboy-2004');
SELECT * FROM public.puntuacion_media;


UPDATE public."Criticas"
SET puntuacion = puntuacion;
SELECT * FROM public.puntuacion_media;
```

Se realizan pruebas para verificar el correcto funcionamiento del cálculo y actualización de la puntuación media.

La primera prueba inserta una nueva crítica y muestra la tabla de puntuación media.

La segunda prueba actualiza todas las críticas y nuevamente muestra la tabla de puntuación media.

Memoria de Usuarios

Tipo de usuarios	Permisos concedidos	Tablas
administrador	ALL PRIVILEGES	Todas
gestor	INSERT, UPDATE, DELETE, SELECT	Todas
critico	SELECT INSERT	Todas Solo la tabla de críticos
cliente	SELECT	Todas

Primer paso, creación de todos los usuarios:

```
CREATE USER administrador WITH PASSWORD 'administrador';
CREATE USER gestor WITH PASSWORD 'gestor';
CREATE USER critico WITH PASSWORD 'critico';
CREATE USER cliente WITH PASSWORD 'cliente';
```

Se crean cuatro tipos de usuarios: administrador, gestor, crítico y cliente.

Cada usuario tiene una contraseña asociada para garantizar la autenticación.

Segundo paso, quitar todos los permisos:

```
REVOKE ALL PRIVILEGES ON DATABASE postgres FROM administrador;
REVOKE ALL PRIVILEGES ON DATABASE postgres FROM gestor;
REVOKE ALL PRIVILEGES ON DATABASE postgres FROM critico;
REVOKE ALL PRIVILEGES ON DATABASE postgres FROM cliente;
```

Se revocan todos los privilegios iniciales en la base de datos postgres para los cuatro usuarios.

Este paso asegura que los usuarios no tengan permisos por defecto.

Tercer paso, dar los privilegios de la tabla:

```
--permisos administrador
GRANT ALL PRIVILEGES ON DATABASE postgres TO administrador;
--permisos gestor
GRANT INSERT, UPDATE, DELETE, SELECT ON ALL TABLES IN SCHEMA public TO
gestor;
--permisos critico
GRANT SELECT ON ALL TABLES IN SCHEMA public TO critico;
GRANT INSERT ON public."Criticas" TO critico;
--permisos cliente
```

```
GRANT SELECT ON ALL TABLES IN SCHEMA public TO cliente;
```

El administrador recibe todos los privilegios en la base de datos.

El gestor tiene permisos de inserción, actualización, eliminación y selección en todas las tablas del esquema público.

El crítico tiene permisos de selección en todas las tablas y permisos de inserción específicamente en la tabla "Criticas".

El cliente tiene permisos de selección en todas las tablas

MODIFICACIONES SOBRE PYTHON

```
import sys
import psycopg2
```

psycopg2 es un adaptador PostgreSQL para Python que permite interactuar con bases de datos PostgreSQL utilizando código Python.

Definimos las variables globales que se usan en el código

```
OPERATION_QUERY = 'query'
OPERATION_INSERT = 'insert'

VALID_USERS = {
    1: ('administrador', 'administrador'),
    2: ('gestor', 'gestor'),
    3: ('critico', 'critico'),
    4: ('cliente', 'cliente'),
    5: ('postgres', 'ISAACSAMUEL')
}
```

```
def ask_conn_parameters():
```

Esta función está diseñada para solicitar al usuario los parámetros de conexión necesarios para establecer una conexión a una base de datos PostgreSQL.

```
for num, (username, _) in VALID_USERS.items():
        print(f'{num}. {username}')

    while True:
        user_choice = get_integer_input('Enter the number for your user:
')
        if user_choice in VALID_USERS:
            break
        else:
            print("Invalid choice. Please select a valid user.")
```

Muestra las opciones de tipo de usuario y solicita al usuario que ingrese su elección.

```
answer1 = input ('¿quieres usar los valores default host = localhost,
port = 5432, database = postgres? (s/n)')
        if (answer1 == 's'):
            host = 'localhost'
            port = 5432
            database = 'postgres'
        else:
            host = input('Introduce el host')
            port = ask_port('TCP port number: ')
            database = input('Introduce el nombre de la base de datos')
```

Si el usuario elige utilizar los valores predeterminados, establece esos valores. De lo contrario, solicita al usuario que ingrese el host, el puerto y el nombre de la base de datos.

```
def menu():
```

Proporciona un menú interactivo para que el usuario elija entre realizar una consulta, insertar datos o salir del programa.

```
print('1. Consulta\n2. Insertar\n0. Salir')

    eleccion = int(input('¿Qué quieres hacer?'))

    if eleccion == 1:
        return input('Introduce la consulta: '), OPERATION_QUERY
    elif eleccion == 2:
        return input('Introduce la crítica: '), OPERATION_INSERT
```

Muestra las opciones del menú (Consulta, Insertar, Salir) y solicita al usuario que ingrese su elección.

Utiliza una estructura condicional para manejar las diferentes opciones del usuario.

Si elige la opción '1', solicita al usuario que ingrese una consulta y devuelve la consulta junto con la etiqueta 'query'.

Si elige la opción '2', solicita al usuario que ingrese una crítica y devuelve la crítica junto con la etiqueta 'insert'.

Si elige la opción '0', sale del programa utilizando sys.exit().

Si la elección no coincide con ninguna opción válida, imprime un mensaje indicando que no se ha introducido una opción válida.

```
def main():

while True:
            cur = conn.cursor()
            operation, operation_type = menu()
            if operation is None:
                break
            try:
                cur.execute(operation)
                if operation_type == OPERATION_QUERY:
                    for record in cur.fetchall():
                        print(record)
                elif operation_type == OPERATION_INSERT:
                    conn.commit()
            except psycopg2.errors.InsufficientPrivilege:
                print("No tienes permisos para hacer esta operación,
intenta con otro usuario")
            except psycopg2.errors.UndefinedTable:
                print("La tabla no existe")
            except psycopg2.errors.SyntaxError:
                print("Error de sintaxis")
            except OperationalError as e:
                print(f"Could not connect to the database: {e}")
                return
            except psycopg2.errors.InFailedSqlTransaction:
                print("The transaction could not be completed.")
                conn.rollback()
            finally:
                cur.close()
    except portException:
        print("The port is not valid!")
    except KeyboardInterrupt:
        print("Program interrupted by user.")
    finally:
        conn.close()
        print("Program finished")
```

1. Solicita los parámetros de conexión a la base de datos a través de la función `ask_conn_parameters()`. Los parámetros de conexión incluyen el host, el puerto, el nombre de usuario, la contraseña y el nombre de la base de datos.

2. Crea una cadena de conexión a partir de los parámetros de conexión y establece una conexión a la base de datos utilizando `psycopg2.connect()`.

3. Entra en un bucle infinito donde:

- Crea un nuevo cursor para la base de datos.
- Llama a la función `menu()` para obtener la operación y el tipo de operación a realizar.
- Si la operación es `None`, rompe el bucle y termina el programa.
- Intenta ejecutar la operación en la base de datos utilizando `cur.execute()`.
- Si el tipo de operación es `OPERATION_QUERY`, recupera y muestra los resultados de la consulta.
- Si el tipo de operación es `OPERATION_INSERT`, confirma la transacción en la base de datos con `conn.commit()`.

4. Maneja varias excepciones que pueden ocurrir durante la ejecución de la operación, como falta de privilegios, tabla no definida, error de sintaxis y error en la transacción. En cada caso, muestra un mensaje de error apropiado.

5. Cierra el cursor después de cada operación, independientemente de si la operación fue exitosa o no.

6. Si se produce una excepción `portException`, muestra un mensaje de error indicando que el puerto no es válido.

7. Si el usuario interrumpe el programa, muestra un mensaje indicando que el programa fue interrumpido por el usuario.

8. Al final del programa, cierra la conexión a la base de datos y muestra un mensaje indicando que el programa ha terminado.